BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN HỌC PHẦN

TÊN ĐỀ TÀI:
XÂY DỰNG MÔ HÌNH PHÂN LOẠI Ý ĐỊNH ĐỂ PHÁT TRIỂN CHATBOT FAQ VỀ QUY CHẾ ĐÀO TẠO TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT

Sinh viên thực hiện:

Đồng Ngọc Phúc – 2221050279

Học phần: Data Mining

Mã môn học: 7080809

Mã nhóm: 02

Giảng viên: TS. Nguyễn Thị Mai Dung

Hà Nội, 2025

MỤC LỤC

MỞ ĐẦU 5
1. Lý do chọn đề tài 5
2. Mục tiêu nghiên cứu 5
3. Phạm vi đề tài 5
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....6
1.1. Tổng quan về bài toán Phân loại văn bản 6
1.2. Tiền xử lý dữ liệu tiếng Việt.....6
1.3. Biểu diễn dữ liệu.....6
1.4. Thuật toán phân loại 8
CHƯƠNG 2: XÂY DỰNG TẬP DỮ LIỆU 10
2.1. Quy trình thu thập và xử lý 10
2.2. Thống kê tập dữ liệu 11
CHƯƠNG 3: HUẤN LUYỆN MÔ HÌNH 12
3.1. Môi trường cài đặt 12
3.2. Tổng quan về quy trình huấn luyện 12
3.3 Tiền xử lý dữ liệu.....12
3.3.1 Input 12
3.3.2 Lowercase 13
3.4 Xử lý dữ liệu 13
3.4.1 Tokenizer 13
3.4.2 Remove Stopwords 13
3.4.3 Phân chia dữ liệu.....15
3.4.4 Sử dụng TF-IDF để số hóa dữ liệu 16
3.5 Bắt đầu huấn luyện 16
CHƯƠNG 4: ĐÁNH GIÁ VÀ CẢI THIỆN MÔ HÌNH 18
4.1 Đánh giá mô hình.....18
4.1.1 Accuracy 18
4.1.2 Confusion Matrix 18

4.1.3 Giới thiệu về TP, FP, FN 20
4.1.4 Precision.....21
4.1.5 Recall 21
4.1.6 F1 - Score.....21
4.2 Cải thiện độ chính xác của mô hình.....22
4.3 Kiểm thử và vấn đề thực tế gặp phải 24
CHƯƠNG 5: ĐÓNG GÓI VÀ TRIỂN KHAI.....27
5.1 Mục đích của việc đóng gói và triển khai mô hình 27
5.2 Giới thiệu các công nghệ đóng gói và triển khai ứng dụng.....27
5.2.1 Joblib.....27
5.2.2 Streamlit.....28
5.3 Quy trình thực hiện.....28
CHƯƠNG 6: TỔNG KẾT 31
6.1 Kết quả đạt được.....31
6.2 Hạn chế 31
6.3 Hướng phát triển.....31
TÀI LIỆU THAM KHẢO 32

MỤC LỤC HÌNH ẢNH

Hình 1. Ví dụ về sản phẩm Chatbot.....5
Hình 2. Ví dụ về tập dữ liệu.....7
Hình 3. Tập dữ liệu JSON 10
Hình 4. Đọc file 12
Hình 5. Kết quả đọc file.....12
Hình 6. Hàm chuyển về chữ thường.....13
Hình 7. Hàm tách từ.....13
Hình 8. Hàm loại bỏ Stopwords 14
Hình 9. Hàm Pipeline tổng hợp các bước xử lý.....14

Hình 10. Áp dụng các hàm xử lý 14
Hình 11. Kết quả sau các bước xử lý 15
Hình 12. In thống kê số lượng mẫu câu 15
Hình 13. Kết quả sau khi in thống kê số lượng mẫu câu 15
Hình 14. Tách dữ liệu thành tập train - test 16
Hình 15. Hiển thị số câu train - test 16
Hình 16. Số hóa dữ liệu bằng TF-IDF 16
Hình 17. Kích thước số liệu sau số hóa 16
Hình 18. Gọi mô hình 17
Hình 19. Sử dụng mô hình 17
Hình 20. Accuracy 18
Hình 21. Confusion Matrix 19
Hình 22. Precision 21
Hình 23. Recall 21
Hình 24. F1-Score 22
Hình 25. Sửa Vectorizer 23
Hình 26. Số hóa lại dữ liệu 23
Hình 27. Huấn luyện lại mô hình 23
Hình 28. Đánh giá mô hình sau khi cải thiện 23
Hình 29. Demo 24
Hình 30. Kết quả demo 24
Hình 31. Khắc phục vấn đề bằng ngưỡng Threshold 25
Hình 32. Kết quả sau khắc phục 25
Hình 33. Điểm yếu của Naïve Bayes 26
Hình 34. Đóng gói mô hình 29
Hình 35. Load model 29
Hình 36. Xử lý đầu vào 30
Hình 37. Tính xác suất 30
Hình 38. Kiểm tra ngưỡng tin cậy 30

Hình 39. Hiển thị câu trả lời 30

MỞ ĐẦU

1. Lý do chọn đề tài

Trong môi trường đại học theo hệ thống tín chỉ, sinh viên thường gặp nhiều khó khăn trong việc nắm bắt các quy định về học vụ, điểm số và điều kiện tốt nghiệp. Các văn bản quy chế, điển hình như Quyết định số 768/QĐ-MĐC, thường có dung lượng dài (hơn 20 trang), ngôn ngữ hành chính phức tạp và khó tra cứu nhanh trên thiết bị di động.

Xuất phát từ nhu cầu thực tế đó, nhóm quyết định thực hiện đề tài "Xây dựng Chatbot tư vấn Quy chế đào tạo". Hệ thống sử dụng các kỹ thuật Khai phá dữ liệu văn bản (Text Mining) để tự động nhận diện câu hỏi của sinh viên và trích xuất câu trả lời chính xác từ văn bản quy chế của nhà trường.

Hình 1. Ví dụ về sản phẩm Chatbot

2. Mục tiêu nghiên cứu

- Xây dựng bộ dữ liệu huấn luyện chuẩn hóa dựa trên Quyết định 768/QĐ-MĐC.
- Áp dụng các kỹ thuật xử lý văn bản để đưa dữ liệu cho máy học.
- Xây dựng mô hình Chatbot có khả năng phản hồi các thắc mắc thường gặp của sinh viên với độ chính xác cao.

3. Phạm vi đề tài

Dữ liệu được giới hạn trong phạm vi "Quy định đào tạo đại học theo hệ thống tín chỉ" (Ban hành ngày 15/09/2021), tập trung vào một vấn đề chính và sinh viên hay thắc mắc: Đăng ký tín chỉ, Khối lượng học tập, Điểm số, Cảnh báo học vụ, Xét tốt nghiệp…

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

1.1. Tổng quan về bài toán Phân loại văn bản

Trước tiên, ta cần phải biết đến khái niệm về Xử lý ngôn ngữ tự nhiên – NLP (Natural Language Processing).

Xử lý ngôn ngữ tự nhiên – NLP (Natural Language Processing) là một nhánh của Trí tuệ nhân tạo, tập trung vào việc nghiên cứu sự tương tác giữa máy tính và ngôn ngữ tự nhiên của con người, dưới dạng tiếng nói (speech) hoặc văn bản (text).

Xử lý ngôn ngữ tự nhiên được chia ra thành hai nhánh lớn, bao gồm:

- Xử lý Tiếng nói (Speech processing).
- Xử lý Văn bản (Text processing).

Trong đó bài toán phân loại văn bản nằm ở nhánh Xử lý văn bản. Phân loại văn bản là bài toán gán nhãn cho một đoạn văn bản đầu vào vào một hoặc nhiều lớp đã định trước. Trong ngữ cảnh của Chatbot, bài toán này trở thành bài toán nhận diện ý định với Input là câu hỏi của người dùng, Output là nhãn chủ đề (ví dụ: Cách tính điểm học phần, Điều kiện xét tốt nghiệp…).

1.2. Tiền xử lý dữ liệu tiếng Việt

Khác với tiếng Anh, tiếng Việt là ngôn ngữ đơn lập, ranh giới từ không được xác định bằng khoảng trắng (ví dụ: "học phí" là một từ ghép, không phải hai từ rời). Do đó, đồ án sử dụng kỹ thuật Tách từ (Tokenization) chuyên dụng cho tiếng Việt bằng thư viện PyVi (Python Vietnamese).

Thư viện PyVi là một bộ công cụ xử lý ngôn ngữ tự nhiên dành riêng cho tiếng Việt, được viết bằng ngôn ngữ lập trình Python.

Mục đích là để nhóm các từ đơn thành từ ghép có nghĩa, nối bằng dấu gạch dưới (VD: sinh viên => sinh_viên).

1.3. Biểu diễn dữ liệu

Để máy tính hiểu được văn bản, nhóm sử dụng phương pháp TF-IDF (Term Frequency - Inverse Document Frequency).

TF-IDF là một kỹ thuật thống kê giúp máy tính đánh giá mức độ quan trọng của một từ trong một văn bản. TF-IDF khá hiệu quả đối với các tập dữ liệu nhỏ và đặc thù nhiều từ khóa chuyên ngành (như "tín chỉ", "cảnh báo học vụ"). Nó giúp làm nổi bật các từ khóa quan trọng đặc trưng cho từng loại câu hỏi.

Máy tính không hiểu chữ viết, nó chỉ hiểu các con số. TF-IDF chính là công cụ giúp biến chữ thành số (vector hóa), nhưng thông minh hơn cách đếm từ thông thường.

TF-IDF được ghép bởi 2 thành phần:

1. TF (Term Frequency - Tần suất xuất hiện): Đo lường tần suất một từ xuất hiện trong một câu cụ thể.

$$\text{TF(t, d)} = \frac{\text{số lần từ t xuất hiện trong văn bản d}}{\text{tổng số từ trong văn bản d}}$$

Ý nghĩa: Từ xuất hiện càng nhiều trong câu đó thì càng quan trọng đối với câu đó.

Trong câu "Học phí này là học phí kỳ 1", từ "học phí" xuất hiện 2 lần → TF cao.

2. IDF (Inverse Document Frequency - Nghịch đảo tần suất văn bản) : Đo lường mức độ phổ biến của từ đó trong toàn bộ dữ liệu (tất cả các câu mẫu).

$$\text{IDF}(t, D) = \log_e \left(\frac{\text{Tổng số văn bản trong tập mẫu } D}{\text{Số văn bản có chứa từ } t} \right)$$

Ví dụ:

Đối với 1 tập dữ liệu gồm nhiều intents (ý định) có cấu trúc:

- tag (nhãn): Mã định danh duy nhất cho ý định hoặc chủ đề đó.
- patterns (mẫu câu): Chứa danh sách các câu hỏi mẫu hay các cách diễn đạt khác nhau của người dùng về chủ đề đó.
- responses: Chứa nội dung câu trả lời chi tiết và chính thức cho ý định (tag) tương ứng. Đây là nội dung mà bot sẽ trả lời người dùng

```
"intents": [
  {
    "tag": "cac_dot_dang_ky_hoc_phan",
    "patterns": [
      "Có mấy đợt đăng ký học phần?",
      "Đăng ký lớp đợt 1 là khi nào?",
      "Đợt 2 đăng ký học phần thời gian thế nào?"
    ],
    "responses": [
      "(Điều 10) Sinh viên đăng ký học phần tại da
    ]
  },
  {
    "tag": "khoi_luong_hoc_tap",
    "patterns": [
```

Hình 2. Ví dụ về tập dữ liệu

Từ nào xuất hiện ở quá nhiều mẫu câu (như "là", "thì", "mà") thì giá trị IDF sẽ thấp (gần bằng 0). Từ nào chỉ xuất hiện ở một nhóm câu nhất định thì IDF sẽ cao.

3. Kết hợp TF và IDF, ta có thể tính TF-IDF theo công thức:

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

- Nếu TF cao (xuất hiện nhiều trong tag đang xét) VÀ IDF cao (hiếm gặp ở tag khác) → Điểm TF-IDF rất cao (Từ khóa vàng).

- Nếu TF cao (xuất hiện nhiều) NHƯNG IDF thấp (từ này tag nào cũng có) → Điểm TF-IDF thấp (Từ phổ thông/Stopword).

1.4. Thuật toán phân loại

Nhóm lựa chọn thuật toán Naive Bayes Classifiers (NBC)

- Naive Bayes Classifiers (NBC) là một trong những thuật toán tiêu biểu cho bài toán phân lớp dựa trên lý thuyết xác suất áp dụng định lý Bayes..
- Định lý Bayes cho phép chúng ta có thể tính toán một xác suất chưa biết dựa vào các xác suất có điều kiện khác. Với công thức tổng quát tính xác suất của biến cố A với điều kiện biến cố Bk xảy ra trước (hay được gọi là xác suất hậu nghiệm). Ví dụ: ”Xác suất trời sẽ mưa nếu hôm nay trời có mây là bao nhiêu?" là một ví dụ về xác suất có điều kiện.

Định lý Bayes có công thức tổng quát:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Trong đó:

- $P(A|B)$: Xác suất hậu nghiệm của biến cố A, biết rằng biến cố B đã xảy ra.
- $P(B|A)$: Xác suất tiên nghiệm của biến cố B, biết rằng biến cố A đã xảy ra.
- $P(A)$: Xác suất của biến cố A.
- $P(B)$: Xác suất của biến cố B.

Trong bài toán chatbot với cấu trúc dữ liệu như đã nói ở ví dụ trong phần TF-IDF trên, chúng ta thay thế các biến cố như sau:

- A (Intent/Class): Là ý định của người dùng (Ví dụ: Chào hỏi, Đặt hàng, Hủy đơn, Khiếu nại).
- B (Feature/Pattern): Là câu nói hoặc các từ khóa người dùng nhập vào (Ví dụ: "Tôi muốn hủy đơn hàng này").

Công thức cụ thể cho chatbot sẽ là:

$$P(\text{Ý định}|Tin\ nhắn) = \frac{P(Tin\ nhắn|\text{Ý định}) * P(\text{Ý định})}{P(Tin\ nhắn)}$$

Giả sử người dùng nhập câu: "Em muốn xin nghỉ học".

Máy sẽ tính toán xác suất xem câu này thuộc về chủ đề nào: Bảo lưu hay Học lại?

1. $P(Tin\ nhắn|\text{Ý định})$ - Likelihood (So sánh với dữ liệu cũ)

Máy sẽ dựa vào dữ liệu đã train và đếm:

- Xét nhóm Bảo lưu:
  - Trong các câu mẫu của nhóm này, từ "nghỉ" xuất hiện rất nhiều (ví dụ: "quy định nghỉ học tạm thời", "thủ tục xin nghỉ").

- Từ "xin" cũng xuất hiện nhiều.

=> Xác suất xuất hiện từ "nghỉ" và “xin” trong nhóm này là CAO.

- Xét nhóm Học lại:
  - Trong nhóm này toàn là từ: "đăng ký", "môn", "trượt", "cải thiện".
  - Hầu như không có từ "nghỉ".

=> Xác suất xuất hiện từ "nghỉ" trong nhóm này là THẤP (gần bằng 0).

2. $P(\text{Ý định})$ - Prior (Độ phổ biến/Ưu tiên)

- Nếu trong tập dữ liệu, nhóm Bảo lưu có 30 câu mẫu.
- Nhóm Học lại chỉ có 10 câu mẫu.

=> Khi máy phân vân, nó sẽ ưu tiên đoán là Bảo lưu vì nó nghĩ chủ đề này phổ biến hơn (Prior cao hơn).

3. $P(\text{Ý định}|Tin\ nhắn)$:

Máy nhân các con số trên lại với nhau:

- Điểm cho nhóm Bảo lưu:
  (Ưu tiên nhóm Bảo lưu) x (Tỉ lệ thấy từ "nghỉ") x (Tỉ lệ thấy từ "xin")
  = rất cao.
- Điểm cho nhóm Học lại: (Ưu tiên nhóm Học lại) x (Tỉ lệ thấy từ "nghỉ" xấp xỉ bằng 0) = rất thấp

=> Máy chọn nhóm Bảo lưu.

4. P(Tin nhắn):

- Vì mẫu số này giống nhau cho mọi ý định nên khi so sánh, ta thường bỏ qua nó.

CHƯƠNG 2: XÂY DỰNG TẬP DỮ LIỆU

2.1. Quy trình thu thập và xử lý

Do file PDF là dạng dữ liệu phi cấu trúc, rất khó để xử lý thông qua việc lập trình và vốn dĩ đề tài chỉ tập trung vào các vấn đề mà sinh viên hay thắc mắc. Nên để cho công việc dễ dàng hơn, ta sẽ cần chuẩn hóa dữ liệu trước.

Quy trình được thực hiện qua 3 bước:

1. **Trích xuất:** Đọc file PDF Quyết định 768/QĐ-MĐC, lọc ra các điều khoản quan trọng nhất.

   Ví dụ: Điều 17 về Cảnh báo kết quả học tập , Điều 23 về Cách tính điểm.

2. **Sinh dữ liệu mẫu:** Tạo ra các biến thể câu hỏi khác nhau từ một quy định gốc, nhằm mô phỏng đa dạng cách hỏi của sinh viên.

   Ví dụ: Từ quy định về cảnh báo học tập, sinh viên có thể hỏi các câu hỏi như "Khi nào thì bị cảnh báo học tập?", "Nếu bị cảnh báo học tập thì sao?"…

3. **Gán nhãn (Labeling):** Dữ liệu được tổ chức dưới dạng JSON gồm cặp patterns (câu hỏi) và tag (nhãn ý định).

File file JSON có cấu trúc như sau:

```
1   {
2     "intents": [
3       {
4         "tag": "cac_dot_dang_ky_hoc_phan",
5         "patterns": [
6           "Có mấy đợt đăng ký học phần?",
7           "Đăng ký lớp đợt 1 là khi nào?",
8           "Đợt 2 đăng ký học phần thời gian thế nào?",
9           "Đợt đăng ký bổ sung là gì?",
10          "Học kỳ phụ có đợt đăng ký bổ sung không?",
11          "Em muốn biết thời gian các đợt đăng ký môn học",
12          "Đăng ký môn trên daotao.humg.edu.vn như thế nào?",
13          "Khi nào là thời điểm điều chỉnh kế hoạch học tập?",
14          "Đợt đăng ký chính thức diễn ra vào lúc nào?",
15          "Thời hạn cuối cùng để đăng ký bổ sung là khi nào?",
16          "Đăng ký học phần được chia thành bao nhiêu giai đoạn?",
17          "Chi tiết thời gian của đợt 1, đợt 2, đợt 3 đăng ký học phần",
18          "Tôi có thể đăng ký môn học ở đâu?",
19          "Thời điểm nào là đăng ký lớp chính thức?",
20          "Đợt đăng ký bổ sung có áp dụng cho học kỳ hè không?",
21          "Đợt đăng ký học phần cuối cùng là đợt nào?",
22          "Làm thế nào để điều chỉnh thời khóa biểu sau đợt 2?",
23          "Khi nào đợt 1 kết thúc?"
24        ],
25        "responses": [
26          "(Điều 10) Sinh viên đăng ký học phần tại daotao.humg.edu.vn và
27        ]
28      },
29      {
30        "tag": "khoi_luong_hoc_tap",
```

Hình 3. Tập dữ liệu JSON

Mảng **intents**: Chứa danh sách tất cả các chủ đề/ý định (Intents) mà người dùng có thể hỏi.

Mỗi phần tử trong mảng "intents" là một đối tượng (Object) đại diện cho một chủ đề cụ thể (ví dụ: Quy định về Tốt nghiệp, Quy định về Tính điểm), và có 3 trường dữ liệu chính:

- tag (nhãn): Mã định danh duy nhất cho ý định hoặc chủ đề đó.
- patterns (mẫu câu): Chứa danh sách các câu hỏi mẫu hay các cách diễn đạt khác nhau của người dùng về chủ đề đó.
- responses: Chứa nội dung câu trả lời chi tiết và chính thức cho ý định (tag) tương ứng. Đây là nội dung mà bot sẽ trả lời người dùng.

2.2. Thống kê tập dữ liệu

Tập dữ liệu bao gồm **10 intents (nhãn)** chính:

1. **[cac_dot_dang_ky_hoc_phan]**: Quy định về thời gian và đặc điểm của 3 đợt đăng ký (chính thức, điều chỉnh, bổ sung) và hướng dẫn thao tác trên hệ thống đào tạo.
2. **[khoi_luong_hoc_tap]**: Giới hạn số tín chỉ tối thiểu và tối đa sinh viên được phép đăng ký trong học kỳ chính và học kỳ phụ, dựa trên xếp loại học lực.
3. **[dang_ky_hoc_lai]**: Các quy định về điều kiện đăng ký học lại để cải thiện điểm (với điểm C, D) và học lại bắt buộc (với điểm F).
4. **[canh_bao_hoc_tap_buoc_thoi_hoc]**: Thông tin về các mức cảnh báo học vụ (Mức 1, 2, 3), quy định hạn chế tín chỉ tương ứng và các trường hợp bị buộc thôi học.
5. **[danh_gia_tinh_diem_hoc_phan]**: Công thức tính điểm tổng kết học phần, bao gồm tỷ trọng điểm quá trình, chuyên cần và thi kết thúc cho cả hình thức trực tiếp và trực tuyến.
6. **[thuc_tap_do_an_tot_nghiep]**: Điều kiện tiên quyết về số tín chỉ và GPA để được nhận đồ án tốt nghiệp, cùng quy trình đăng ký thực tập doanh nghiệp.
7. **[dieu_kien_xet_tot_nghiep]**: Các tiêu chuẩn đầu ra bắt buộc (Ngoại ngữ, Tin học, GDQP...) và điều kiện về điểm trung bình tích lũy để được công nhận tốt nghiệp.
8. **[chuyen_nganh_chuyen_doi]**: Quy định chuyển ngành, chuyển trường và chuyển đổi hình thức học.
9. **[bao_luu_thoi_hoc]**: Quy định nghỉ học tạm thời và bảo lưu kết quả.
10. **[xep_hang_hoc_tap]**: Xếp hạng học tập và tốt nghiệp dựa trên Điểm TBC tích lũy toàn khóa (thang điểm 4)

CHƯƠNG 3: HUẤN LUYỆN MÔ HÌNH

3.1. Môi trường cài đặt

- **Ngôn ngữ:** Python.
- **Thư viện chính:** scikit-learn (Huấn luyện mô hình), pyvi (Xử lý tiếng Việt), pandas (Xử lý dữ liệu).
- **Công cụ:** Jupyter Notebook.

3.2. Tổng quan về quy trình huấn luyện

1. **Input:** Chuỗi văn bản thô.
2. **Lowercase:** Đưa mọi ký tự về dạng viết thường.
3. **Tokenizer:** Tách từ tiếng Việt (ViTokenizer).
4. **Remove Stopwords:** Xóa những từ không mang nhiều ý nghĩa.
5. **Phân chia dữ liệu:** Tách dữ liệu thành tập Train - Test
6. **Vectorizer:** Chuyển đổi sang vector TF-IDF.
7. **Classifier:** Phân loại bằng Naive Bayes.

3.3 Tiền xử lý dữ liệu

3.3.1 Input

Trước tiên cần tải dữ liệu từ tệp JSON và kiểm tra sơ bộ dữ liệu để đảm bảo quá trình đọc tệp đã thành công.

```
import json

# Đọc file dữ liệu
DATA_PATH = 'data/data_daotao.json'

with open(DATA_PATH, 'r', encoding='utf-8') as f:
    raw_data = json.load(f)

# Kiểm tra thử xem đọc được chưa bằng cách in ra 1 intent đầu tiên
print("Mẫu dữ liệu đầu tiên:", raw_data['intents'][0])
```

[1]

Hình 4. Đọc file

Thu được kết quả:

```
Mẫu dữ liệu đầu tiên: {'tag': 'cac_dot_dang_ky_hoc_phan', 'patterns': ['Có mấy đợt đăng ký học phần?',
```

Hình 5. Kết quả đọc file

Do khi nhận câu hỏi từ người dùng, ta cũng cần phải xử lý các dữ liệu đầu vào đó nên ta sẽ xây dựng các hàm dành cho các bước tiền xử lý để tái sử dụng sau này

3.3.2 Lowercase

Đưa mọi ký tự về dạng viết thường để đảm bảo tính nhất quán

```
def step1_lowercase(text):
    return text.lower()
```

[2]

Hình 6. Hàm chuyển về chữ thường

3.4 Xử lý dữ liệu

3.4.1 Tokenizer

Tách từ (Tokenization) là kỹ thuật giúp máy tính xác định đâu là một từ có nghĩa trọn vẹn trong câu, thay vì chỉ cắt câu dựa trên các dấu cách (khoảng trắng).

Đây là bước quan trọng nhất khi xử lý Tiếng Việt vì đặc thù ngôn ngữ của chúng ta khác với Tiếng Anh.

- **Tiếng Anh**: Đa phần các từ được phân tách nhau bằng dấu cách. Ví dụ: "I love football" → Máy cắt ngay được 3 từ: ["I", "love", "football"].
- **Tiếng Việt**: Dấu cách dùng để tách âm tiết (tiếng), không phải tách từ. Từ trong tiếng Việt có thể là từ đơn (1 chữ) hoặc hoặc từ ghép (2, 3, 4 chữ).

```
from pyvi import ViTokenizer  # Thư viện tách từ tiếng Việt
def step2_tokenize(text):
    return ViTokenizer.tokenize(text)
```

[3]

Hình 7. Hàm tách từ

3.4.2 Remove Stopwords

StopWords là những từ xuất hiện nhiều trong ngôn ngữ tự nhiên, tuy nhiên lại không mang nhiều ý nghĩa. Ở tiếng việt StopWords là những từ như: để, này, kia...

Do các thư viện về xử lý ngôn ngữ Tiếng Việt chưa được phổ biến và mạnh mẽ, cộng thêm việc quy mô của đề tài không quá lớn. Việc tạo sẵn các danh sách Stopwords sẽ đủ tốt và dễ kiểm soát hơn.

```python
vietnamese_stopwords = set([
    "là", "của", "các", "những", "cho", "với", "thì", "mà", "bị", "bởi", "được", "tại", "vì", "rằng",
    "em", "muốn", "biết", "ạ", "ơi", "hỏi", "về", "như", "này", "khi", "trong", "để", "làm", "gì",
    "nào", "ở", "có", "không", "nhưng", "rất", "cũng", "đã", "sẽ", "đang", "vẫn", "cứ", "chỉ",
    "nếu", "hoặc", "hay", "do", "nên", "thôi", "nữa", "đâu", "đấy", "đây", "rồi", "chi", "nhiêu",
    "để", "này", "kia"
])
def step3_remove_stopwords(text, stopwords = vietnamese_stopwords):
    # Tách chuỗi thành danh sách các từ
    words = text.split()
    # Lọc bỏ từ nằm trong danh sách stopwords
    filtered_words = [w for w in words if w not in stopwords]
    # Ghép lại thành chuỗi
    return " ".join(filtered_words)
```

[4]

Hình 8. Hàm loại bỏ Stopwords

Tạo pipeline để tổng hợp các bước trên

```python
def processing_pipeline(text):
    if not text: return ""

    text = step1_lowercase(text)
    text = step2_tokenize(text)
    text = step3_remove_stopwords(text)
    return text
```

[2]

Hình 9. Hàm Pipeline tổng hợp các bước xử lý

Áp dụng vào dữ liệu huấn luyện

```python
import pandas as pd  # Thư viện xử lý bảng dữ liệu
corpus = []  # Chứa câu hỏi đã xử lý
labels = []  # Chứa nhãn (Intent)

# Duyệt qua dữ liệu JSON
for intent in raw_data['intents']:
    for pattern in intent['patterns']:

        # Gọi hàm tổng hợp để xử lý từng câu mẫu
        cleaned_text = processing_pipeline(pattern)

        # Chỉ thêm vào nếu câu không bị rỗng sau khi lọc
        if cleaned_text:
            corpus.append(cleaned_text)
            labels.append(intent['tag'])

# Tạo DataFrame
df = pd.DataFrame({'text': corpus, 'intent': labels})

print("Dữ liệu sau khi qua các bước xử lý riêng biệt:")
display(df.head(10))
```

[ ]

Hình 10. Áp dụng các hàm xử lý

Dữ liệu sau khi qua các bước xử lý riêng biệt:

	text	intent
0	mấy đợt đăng_ký học_phần ?	cac_dot_dang_ky_hoc_phan
1	đăng_ký lớp đợt 1 ?	cac_dot_dang_ky_hoc_phan
2	đợt 2 đăng_ký học_phần thời_gian thế_nào ?	cac_dot_dang_ky_hoc_phan
3	đợt đăng_ký bổ_sung ?	cac_dot_dang_ky_hoc_phan
4	học_kỳ phụ đợt đăng_ký bổ_sung ?	cac_dot_dang_ky_hoc_phan
5	thời_gian đợt đăng_ký môn_học	cac_dot_dang_ky_hoc_phan
6	đăng_ký môn trên daotao . humg . edu . vn thế_...	cac_dot_dang_ky_hoc_phan
7	thời_điểm điều_chỉnh kế_hoạch học_tập ?	cac_dot_dang_ky_hoc_phan
8	đợt đăng_ký chính_thức diễn ra vào lúc_nào ?	cac_dot_dang_ky_hoc_phan
9	thời_hạn cuối_cùng đăng_ký bổ_sung ?	cac_dot_dang_ky_hoc_phan

Hình 11. Kết quả sau các bước xử lý

```
# In thống kê số lượng cụ thể
print("Thống kê số lượng mẫu câu:")
print(df['intent'].value_counts())
```

[7]

Hình 12. In thống kê số lượng mẫu câu

```
Thống kê số lượng mẫu câu:
intent
dieu_kien_xet_tot_nghiep          31
cac_dot_dang_ky_hoc_phan          30
thuc_tap_do_an_tot_nghiep         30
chuyen_nganh_chuyen_doi           30
bao_luu_thoi_hoc                  30
xep_hang_hoc_tap                  30
khoi_luong_hoc_tap                29
dang_ky_hoc_lai                   29
canh_bao_hoc_tap_buoc_thoi_hoc    29
danh_gia_tinh_diem_hoc_phan       29
Name: count, dtype: int64
```

Hình 13. Kết quả sau khi in thống kê số lượng mẫu câu

3.4.3 Phân chia dữ liệu

Tách dữ liệu thành tập Train – Test

```python
from sklearn.model_selection import train_test_split

# Chia dữ liệu: 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
    df['text'],     # Đầu vào (câu hỏi)
    df['intent'],   # Đầu ra (nhãn)
    test_size=0.2, # 20% để test
    random_state=42 # Giữ cố định cách chia để kết quả giống nhau mỗi lần chạy
)

print(f"Số câu để huấn luyện: {len(X_train)}")
print(f"Số câu để kiểm thử: {len(X_test)}")
```

[8]

Hình 14. Tách dữ liệu thành tập train - test

```
Số câu để huấn luyện: 237
Số câu để kiểm thử: 60
```

Hình 15. Hiển thị số câu train - test

3.4.4 Sử dụng TF-IDF để số hóa dữ liệu

Phần này nhóm sử dụng lớp TfidVectorizer lấy từ thư viện sklearn.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
# Khởi tạo TF-IDF
vectorizer = TfidfVectorizer()

# 1. Học từ vựng từ tập Train và biến nó thành số
# fit: Học từ vựng (tạo từ điển)
# transform: Biến đổi chữ thành số
X_train_vec = vectorizer.fit_transform(X_train)

# 2. Chỉ biến đổi tập Test thành số (Dùng từ điển đã học ở trên)
X_test_vec = vectorizer.transform(X_test)

print(f"Kích thước dữ liệu số hóa: {X_train_vec.shape}")  # (Số câu, Số lượng từ vựng máy đã học được)
```

[9]

Hình 16. Số hóa dữ liệu bằng TF-IDF

```
Kích thước dữ liệu số hóa: (237, 311)
```

Hình 17. Kích thước số liệu sau số hóa

3.5 Bắt đầu huấn luyện

Trong thư viện Scikit-learn, có 3 loại Naive Bayes chính:

- GaussianNB: Dùng cho dữ liệu là số thực liên tục (ví dụ: chiều cao, cân nặng).
- BernoulliNB: Dùng cho dữ liệu nhị phân (có/không).
- MultinomialNB: Dùng cho dữ liệu dạng đếm số lượng (tần suất).

Vì chúng ta dùng TF-IDF (bản chất là đếm tần suất từ xuất hiện), nên MultinomialNB là sự lựa chọn chuẩn nhất cho xử lý văn bản.

```
from sklearn.naive_bayes import MultinomialNB
model = MultinomialNB()
#Danh sách các tham số của model:
MultinomialNB().get_params()
```

[11] ✓ 0.0s

```
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}
```

Hình 18. Gọi mô hình

Trong hàm MultinomialNB(), có các tham số ta cần quan tâm để áp dụng vào bài toán này:

- alpha (Tham số làm trơn): Cộng thêm một con số nhỏ vào tần suất của mọi từ để đảm bảo không có từ nào có xác suất bằng 0. Trong phép nhân, nếu một thành phần bằng 0 thì toàn bộ kết quả bằng 0. Máy sẽ bị lỗi hoặc dự đoán sai.
- fit_prior (Học xác suất ưu tiên): Cho phép máy tự học xem chủ đề (Intent) nào phổ biến hơn.
- class_prior (Gán cứng độ phổ biến): Dùng để ép buộc máy phải tin vào một tỷ lệ xác suất do ta tự bịa ra, thay vì học từ dữ liệu.
- force_alpha (Kỹ thuật): Nếu alpha về một số cực nhỏ (gần bằng 0), máy tính có thể gặp lỗi làm tròn số.

```
# Đưa dữ liệu số hóa (X_train_vec) và đáp án (y_train) vào để máy học
model.fit(X_train_vec, y_train)
```

[12] ✓ 0.0s

```
MultinomialNB
MultinomialNB()
```

Hình 19. Sử dụng mô hình

CHƯƠNG 4: ĐÁNH GIÁ VÀ CẢI THIỆN MÔ HÌNH

4.1 Đánh giá mô hình

Mô hình sẽ được đánh giá dựa trên các tham số sau:

Accuracy, Confusion Matrix, Precision, Recall, F1-score

4.1.1 Accuracy

Đo lường tỷ lệ các mẫu dự đoán đúng so với tổng số mẫu trong tập.

```
from sklearn.metrics import accuracy_score
# Dự đoán thử trên tập Test (20% dữ liệu đã giấu)
y_pred_nb = model.predict(X_test_vec)

acc_nb = accuracy_score(y_test, y_pred_nb)
#y_test: danh sách các nhãn đúng của 20% dữ liệu đã "cất đi" lúc chia train_test_split.

print(f"Độ chính xác của mô hình trên tập Test: {acc_nb*100:.2f}%")

acc_nb2 = accuracy_score(y_test, y_pred_nb, normalize=False)
print('Tổng số mẫu dự đoán đúng:', acc_nb2, ' /', len(y_test))
print('Tổng số mẫu dự đoán sai:', len(y_test) - acc_nb2, ' /', len(y_test))
```

✓ 0.0s

```
Độ chính xác của mô hình trên tập Test: 81.67%
Tổng số mẫu dự đoán đúng: 49.0  / 60
Tổng số mẫu dự đoán sai: 11.0  / 60
```

Hình 20. Accuracy

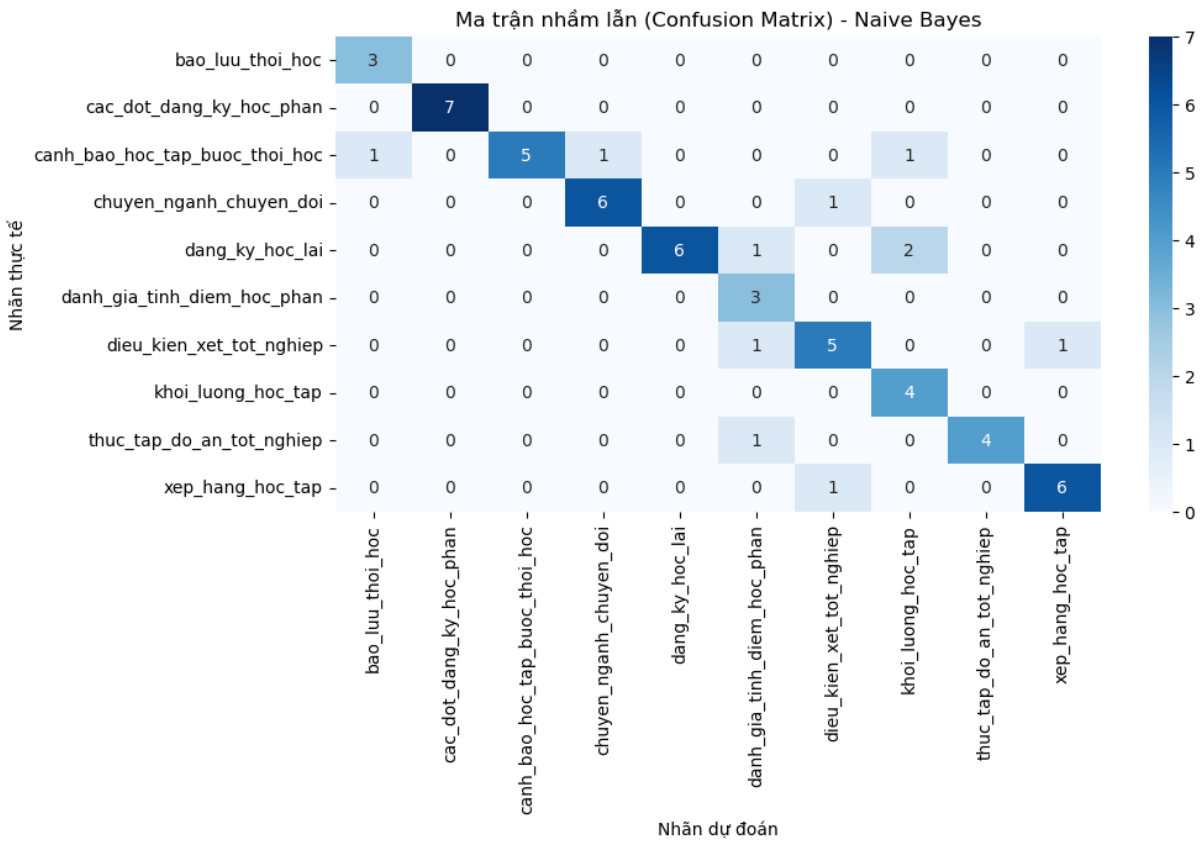
4.1.2 Confusion Matrix

Confusion Matrix (Ma trận Nhầm lẫn) là một bảng báo cáo chi tiết giúp ta "bắt bệnh" cho Chatbot. Nó cho ta biết chính xác model đang bị nhầm lẫn ở đâu, chứ không chỉ đưa ra một con số điểm tổng quát.

Cấu trúc của Ma trận:

- Hàng ngang (Rows): Là nhãn Thực tế (Người dùng thực sự hỏi gì).
- Cột dọc (Columns): Là nhãn Máy đoán (Bot nghĩ là cái gì).

Mục tiêu: Chúng ta muốn các con số lớn nhất đều nằm trên Đường chéo chính (từ góc trên trái xuống góc dưới phải). Đó là những lần Bot đoán đúng.



Hình 21. Confusion Matrix

- Đường chéo chính (Màu xanh đậm): Các ô nằm trên đường chéo từ góc trên trái xuống góc dưới phải thể hiện số lần dự đoán ĐÚNG. Nhìn chung, đường chéo này có màu đậm nhất, chứng tỏ mô hình hoạt động khá ổn định (khoảng trên 80%).

- Các ô ngoài đường chéo (Màu nhạt/Trắng): Thể hiện những lần dự đoán SAI. Có một số điểm nhầm lẫn đáng chú ý cần khắc phục.

Những chủ đề (Intent) mà mô hình nhận diện khá chuẩn, hầu như không sai sót:

cac_dot_dang_ky_hoc_phan:

- Thực tế: 7 câu.

- Đoán đúng: 7/7 (100%).

bao_luu_thoi_hoc:

- Thực tế: 3 câu.

- Đoán đúng: 3/3 (100%).

- Lưu ý: Tuy nhiên, nó lại bị các nhóm khác nhận nhầm sang 1 lần.

Nhóm cần cải thiện:

Chủ đề dang_ky_hoc_lai (Đăng ký học lại)

- Vấn đề: Thực tế có 9 câu, chỉ đúng 6.

- Nhầm sang: khoi_luong_hoc_tap (2 lần), danh_gia_tinh_diem... (1 lần).

- Phân tích nguyên nhân: Có thể trong câu hỏi học lại có chứa từ "tín chỉ" hoặc "điểm" (ví dụ: "điểm thấp có phải học lại không?"). Vì Naive Bayes đếm từ, nên nó thấy chữ "điểm" thì nghĩ là danh_gia_tinh_diem, thấy "tín chỉ" thì nghĩ là khoi_luong.

Chủ đề canh_bao_hoc_tap_buoc_thoi_hoc (Cảnh báo/Buộc thôi học)

- Vấn đề: Đây là chủ đề bị phân tán dữ liệu nhiều nhất (đúng 5/8).

- Nhầm sang:

  - bao_luu_thoi_hoc (1 lần): Dễ hiểu vì cùng có từ "thôi học", "nghỉ học".

  - chuyen_nganh... (1 lần).

  - khoi_luong_hoc_tap (1 lần).

Chủ đề dieu_kien_xet_tot_nghiep

- Vấn đề: Đúng 5/7.

- Nhầm sang: danh_gia_tinh_diem... (1 lần) và xep_hang_hoc_tap (1 lần).

- Phân tích: Các chủ đề này đều liên quan nặng đến "điểm số" (GPA, điểm tổng kết). Máy đang bị loạn giữa các loại điểm.

4.1.3 Giới thiệu về TP, FP, FN

TP (True Positive) có thể hiểu là số lần Bot đoán đúng so với thực tế.

- TP = 6. Đây là con số nằm ngay trên đường chéo chính màu đậm, tại giao điểm của hàng dang_ky_hoc_lai và cột dang_ky_hoc_lai.

- Có 6 câu hỏi thực tế là về "Học lại" và Bot đã đoán đúng là "Học lại".

FN (False Negative) là số lần Bot đoán sai

- Nhìn vào Hàng Ngang của dang_ky_hoc_lai. Ngoài số 6 (đúng) ra, ta thấy còn có số 1 (ở cột danh_gia_tinh_diem) và số 2 (ở cột khoi_luong_hoc_tap).

- Tổng FN = 1 + 2 = 3.

=>

- Có 3 câu hỏi thực tế là "Học lại", nhưng Bot đã bỏ sót.

- Cụ thể: 1 câu bị Bot đoán nhầm sang "Tính điểm", 2 câu bị đoán nhầm sang "Khối lượng học tập".

FP (False Positive) là Bot "nghĩ mình" là đúng nhưng thực tế là sai

- Nhìn vào Cột Dọc của dang_ky_hoc_lai. Ngoài số 6 (đúng) ra, ta nhìn dọc lên trên và xuống dưới xem có số nào khác không.

- Trong ảnh, cột này toàn số 0.

- Tổng FP = 0. Chứng tỏ không có câu hỏi nào của chủ đề khác bị Bot đoán nhầm thành "Học lại".

4.1.4 Precision

Precision (Độ chính xác) là tỷ lệ số điểm True Positive (TP) trong số những điểm được phân loại là Positive (TP+FP). Như vậy Precision càng cao, độ chính xác trong kết luận của mô hình càng cao.

```
from sklearn.metrics import precision_score

# Tính Precision (Độ chính xác dự đoán)
# Lưu ý: Phải thêm average='weighted' vì đây là bài toán nhiều lớp (Multi-class)
prec_nb = precision_score(y_test, y_pred_nb, average='weighted')

print(f"Độ chính xác dự đoán (Precision): {prec_nb*100:.2f}%")
```
✓ 0.0s

```
Độ chính xác dự đoán (Precision): 86.73%
```

Hình 22. Precision

4.1.5 Recall

Recall (Độ bao phủ) được định nghĩa tỷ lệ số điểm True Positive (TP) trong tổng số Positive thực sự (TP+FN). Như vậy Recall càng cao thì mô hình quét càng nhạy, không để sót bất kì trường hợp nào.

```
from sklearn.metrics import recall_score

# Tính Recall (Độ bao phủ)
# average='weighted': Tính trung bình có trọng số cho bài toán nhiều lớp
rec_nb = recall_score(y_test, y_pred_nb, average='weighted')

print(f"Độ bao phủ (Recall): {rec_nb*100:.2f}%")
```
✓ 0.0s

```
Độ bao phủ (Recall): 81.67%
```

Hình 23. Recall

4.1.6 F1 - Score

Nếu chỉ dùng Precision, mô hình chỉ đưa ra dự đoán cho một điểm mà nó chắc chắn nhất. Khi đó Precision = 1, tuy nhiên ta không thể nói là mô hình này tốt.

Nếu chỉ dùng Recall, nếu mô hình dự đoán tất cả các điểm đều là positive. Khi đó Recall = 1, tuy nhiên ta cũng không thể nói đây là mô hình tốt.

=> Khi đó F1-score được sử dụng. F1-score là trung bình điều hòa của precision và recall. **F1 = 2 × (Precision × Recall) / (Precision + Recall)**

```
from sklearn.metrics import f1_score

# Tính F1-Score (Trung bình điều hòa giữa Precision và Recall)
# average='weighted': Quan trọng cho bài toán nhiều lớp
f1_nb = f1_score(y_test, y_pred_nb, average='weighted')

print(f"Điểm F1-Score: {f1_nb*100:.2f}%")
✓ 0.0s
```

```
Điểm F1-Score: 82.13%
```

Hình 24. F1-Score

4.2 Cải thiện độ chính xác của mô hình

Dựa trên các đánh giá về độ chính xác, ta có thể cải thiện độ chính xác của mô hình qua những các sau:

1. Bổ sung Stopwords: Thêm các từ điển vào danh sách stopwords để loại bỏ bớt nhiễu. Nếu không loại bỏ, mô hình Naive Bayes (vốn dựa trên xác suất) sẽ gây sai số lớn.

2. Tăng cường từ khóa đặc trưng: Đây là 2 cách trực tiếp nhất để sửa lỗi Recall thấp

- Với dang_ky_hoc_lai: Thêm nhiều mẫu câu chứa từ "trượt", "rớt môn", "học lại", "cải thiện".

- Với canh_bao...: Thêm từ "đuổi học", "cảnh cáo", "mức".

3. Thử nghiệm Bigram (Cụm 2 từ):

- Thay đổi ngram_range=(1, 2) trong TfidfVectorizer.

- Lý do: Để máy phân biệt được "thôi học" (trong buộc thôi học) và "nghỉ học" (trong bảo lưu). Hiện tại máy đang tách rời các từ này nên dễ nhầm.

Cách 1 và cách 2 đều là cách làm thủ công và phải dựa vào các phản hồi từ người dùng khi Bot trả lời sai nên ta sẽ triển khai ngay phương pháp thứ 3 vì nó khả thi ngay bây giờ.

Bước 1: Sửa Vectorizer

```
# BƯỚC 1: SỬA VECTORIZER
# ngram_range=(1, 2): Có nghĩa là học cả từ đơn (Unigram) VÀ cụm 2 từ (Bigram)
# Ví dụ: "học máy" -> Máy sẽ học 3 thứ: "học", "máy", "học máy"
vectorizer_bi = TfidfVectorizer(ngram_range=(1, 2))
```

Hình 25. Sửa Vectorizer

Bước 2: Số hóa lại dữ liệu

```
# BƯỚC 2: SỐ HÓA DỮ LIỆU LẠI TỪ ĐẦU
# Học lại từ vựng mới (bao gồm cả cụm từ)
X_train_bi = vectorizer_bi.fit_transform(X_train)
X_test_bi = vectorizer_bi.transform(X_test)

print(f"Số lượng từ vựng cũ (Unigram): {X_train_vec.shape[1]}") # Biến cũ ở các bước trước
print(f"Số lượng từ vựng mới (Bigram): {X_train_bi.shape[1]}")  # Biến mới
✓ 0.0s

Số lượng từ vựng cũ (Unigram): 311
Số lượng từ vựng mới (Bigram): 1116
```

Hình 26. Số hóa lại dữ liệu

Bước 3: Huấn luyện lại mô hình

```
# BƯỚC 3: HUẤN LUYỆN LẠI MODEL
model_bi = MultinomialNB()
model_bi.fit(X_train_bi, y_train)
✓ 0.0s

▾ MultinomialNB
MultinomialNB()
```

Hình 27. Huấn luyện lại mô hình

Bước 4: Đánh giá kết quả

```
##BƯỚC 4: ĐÁNH GIÁ KẾT QUẢ MỚI
y_pred_bi = model_bi.predict(X_test_bi)

#Độ chính xác cũ:
print(f"Độ chính xác cũ (Unigram): {acc_nb*100:.2f}%")

acc_bi = accuracy_score(y_test, y_pred_bi)
print(f"Độ chính xác (sau khi áp dụng Bigram): {acc_bi*100:.2f}%")

✓ 0.0s

Độ chính xác cũ (Unigram): 81.67%
Độ chính xác (sau khi áp dụng Bigram): 85.00%
```

Hình 28. Đánh giá mô hình sau khi cải thiện

Dù không quá nhiều nhưng phương pháp Bigram đã có hiệu quả và giúp tăng độ chính xác cho mô hình.

4.3 Kiểm thử và vấn đề thực tế gặp phải

```
# NHẬP INPUT ĐỂ TEST
input_text = "Bot ăn cơm chưa"

#XỬ LÝ

# Bước 1: Gọi hàm pipline để xử lý input
processed_input = processing_pipeline(input_text)
# Biến đổi chữ thành số (Vectorize)
input_vec = vectorizer_bi.transform([processed_input])

# Bước 2: Dự đoán Intent
pred_intent = model_bi.predict(input_vec)[0]

# Bước 3: Lấy câu trả lời
if pred_intent in response_map:
    # Lấy phần tử đầu tiên [0] trong list câu trả lời
    response_text = response_map[pred_intent][0]
else:
    response_text = "Xin lỗi, bot chưa học câu này."

# 4. IN KẾT QUẢ RA MÀN HÌNH

print(f"INPUT (Người dùng nhập):  {input_text}")
print(f"TOKENIZED (Máy đã tách):  {processed_input}")
print("-" * 60)
print(f"INTENT (Dự đoán):         [{pred_intent}]")
print(f"RESPONSE (Bot trả lời):    {response_text}")
```

Hình 29. Demo

```
INPUT (Người dùng nhập):  Bot ăn cơm chưa
DỮ LIỆU ĐƯỢC ĐƯA VÀO MÁY:  bot ăn cơm chưa
------------------------------------------------------------
INTENT (Dự đoán):         [thuc_tap_do_an_tot_nghiep]
RESPONSE (Bot trả lời):    (Điều 26 & 27) Quy định thực tập và đồ án tốt nghiệp:
- Điều kiện đi thực tập tốt nghiệp (doanh nghiệp): Hoàn thành hầu hết học phần (trừ đồ án
- Đăng ký và Hướng dẫn: Đăng ký theo kế hoạch ngành/chuyên ngành. Mỗi sinh viên có giảng
- Đánh giá: Điểm TTTN/TTDN: Điểm quá trình do giảng viên hướng dẫn dựa trên nhận xét cơ s
- Thủ tục: Phòng ĐTĐH xét và công bố danh sách sinh viên đủ điều kiện nhận đồ án tốt nghi
```

Hình 30. Kết quả demo

Theo như đoạn test trên, ta đã gặp phải trường hợp "False Positive" (Dương tính giả) trong thực tế.

Nguyên nhân:

- **Từ lạ:** Từ "Bot ăn cơm chưa" không có trong tập dữ liệu huấn luyện (JSON) của các Intent hiện tại. Khi vectorizer gặp từ lạ, nó tạo ra vector rỗng/gần rỗng.

- **Cơ chế Naive Bayes:** Khi gặp input lạ hoặc không rõ ràng, thuật toán thường có xu hướng "đoán bừa" vào nhãn (intent) nào có số lượng câu mẫu nhiều nhất trong tập Train (hoặc nhãn có xác suất tiên nghiệm cao nhất). Có vẻ như nhãn thuc_tap_do_an_tot_nghiep đang chiếm ưu thế.

Ta có thể khắc phục vấn đề này bằng cách lọc bỏ câu có độ tin cậy thấp:

- Đây là cách quan trọng nhất để chặn các câu nhập linh tinh (ví dụ: "dghdfg", "ăn cơm chưa",...).

- Thay vì dùng model.predict() (chỉ trả về nhãn), ta dùng model.predict_proba() (trả về xác suất % của từng nhãn). Nếu xác suất cao nhất mà vẫn thấp (ví dụ dưới 20% hoặc 10%), ta coi như Bot không hiểu.

```
# BƯỚC QUAN TRỌNG: TÍNH XÁC SUẤT
# Thay vì chỉ predict, ta lấy xác suất (probability)
probs = model_bi.predict_proba(input_vec)[0] # Lấy danh sách % của tất cả các nhãn
max_prob = max(probs)                         # Lấy con số % cao nhất
pred_index = probs.argmax()                   # Lấy vị trí của nhãn cao nhất
pred_intent = model_bi.classes_[pred_index]   # Lấy tên nhãn (tag)


# Bước 3: Chọn câu trả lời dựa trên độ tin cậy
if max_prob > THRESHOLD: #Threshold = 0.15
    # Trường hợp 1: >15%
    if pred_intent in response_map:
        response_text = response_map[pred_intent][0]
    else:
        response_text = "Xin lỗi, mình gặp lỗi dữ liệu (có nhãn nhưng không có câu trả lời)."
else:
    # Trường hợp 2: Không rõ (<15%)
    response_text = "Xin lỗi, câu này mình chưa được học hoặc bạn nhập chưa rõ nghĩa."

# 4. IN KẾT QUẢ RA MÀN HÌNH
print(f"INPUT:            {input_text}")
print(f"DỮ LIỆU ĐƯỢC ĐƯA VÀO MÁY:     {processed_input}")
print("-" * 60)
print(f"INTENT (Dự đoán): [{pred_intent}]")
print(f"ĐỘ TIN CẬY:       {max_prob*100:.2f}%")
print("-" * 60)
print(f"RESPONSE:         {response_text}")
```

Hình 31. Khắc phục vấn đề bằng ngưỡng Threshold

```
INPUT:            Bot ăn cơm chưa
DỮ LIỆU ĐƯỢC ĐƯA VÀO MÁY:     bot ăn cơm chưa
------------------------------------------------------------
INTENT (Dự đoán): [thuc_tap_do_an_tot_nghiep]
ĐỘ TIN CẬY:       13.36%
------------------------------------------------------------
RESPONSE:         Xin lỗi, câu này mình chưa được học hoặc bạn nhập chưa rõ nghĩa.
```

Hình 32. Kết quả sau khắc phục

Tuy nhiên vẫn có một điểm yếu chí mạng của mô hình Naïve Bayes do đây là mô hình dựa trên xác suất:

```
INPUT:            Cách tính điểm học phần
DỮ LIỆU ĐƯỢC ĐƯA VÀO MÁY:     cách tính điểm học_phần
------------------------------------------------------------
INTENT (Dự đoán): [danh_gia_tinh_diem_hoc_phan]
ĐỘ TIN CẬY:       35.24%
------------------------------------------------------------
RESPONSE:         (Điều 20) Quy định đánh giá và tính điểm học
- Học phần lý thuyết: điểm học phần được tính dựa trên điểm ki
  * Học trực tiếp: Điểm Học phần = 0,6*A + 0,3*B + 0,1*C.
  * Học trực tuyến: Điểm Học phần = 0.5*A + 0,4*B + 0,1*C.
```

Hình 33. Điểm yếu của Naïve Bayes

Tại sao "Cách tính điểm học phần" đúng chủ đề nhưng chỉ được 35.24%?

Trường hợp này gọi là “Phân tán trọng số”. Câu hỏi chứa quá nhiều từ khóa "phổ thông" xuất hiện ở nhiều chủ đề khác nhau, khiến máy bị phân vân. Đó cũng là lý do nhóm phải đặt ngưỡng tin cậy rất thấp (15%).

Thay vì dồn 90% niềm tin vào một chỗ, máy phải chia sẻ niềm tin cho nhiều nhóm. Ví dụ nó tính toán ngầm như sau:

- Nhóm danh_gia_tinh_diem_hoc_phan: **35.24%** (Cao nhất vì khớp được cả "tính" và "điểm").
- Nhóm dang_ky_hoc_phan: **25%** (Vì có chữ "học phần").
- Nhóm xep_hang_hoc_tap: **20%** (Vì có chữ "điểm").
- Các nhóm khác: **...%**.

Ta có thể khắc phục bằng 2 cách chính:

- Bổ sung mẫu này câu cho nhóm " danh_gia_tinh_diem _hoc_phan". Điều này sẽ giúp tỷ lệ được tăng lên đáng kể.
- Cải thiện chất lượng mẫu câu ở tất cả các intent, hạn chế hiện tượng lặp từ với các intent khác và cải thiện từ điển stopwords cho hợp lý hơn.

CHƯƠNG 5: ĐÓNG GÓI VÀ TRIỂN KHAI

5.1 Mục đích của việc đóng gói và triển khai mô hình

Việc chuyển đổi từ mã nguồn huấn luyện sang ứng dụng thực tế đòi hỏi hai công đoạn quan trọng là **Đóng gói** và **Triển khai.**

Tối ưu hóa hiệu năng và tài nguyên:

- **Tránh phải huấn luyện lặp lại:** Quá trình huấn luyện mô hình tốn nhiều thời gian và tài nguyên tính toán (CPU/RAM). Việc lưu mô hình giúp "đóng băng" trạng thái "trí tuệ" của máy sau khi học xong. Khi cần sử dụng, hệ thống chỉ cần tải lại thay vì phải chạy lại thuật toán từ đầu.
- **Đảm bảo tính nhất quán:** Việc lưu trữ cả Vectorizer (bộ chuyển đổi văn bản sang số) và Model (thuật toán) vào cùng một file đảm bảo rằng dữ liệu đầu vào thực tế sẽ được xử lý y hệt như lúc huấn luyện, tránh sai lệch dữ liệu.

Tăng tính tương tác và ứng dụng:

- **Giao diện hóa:** Mô hình máy học chỉ là các ma trận số học, người dùng cuối (sinh viên) không thể tương tác trực tiếp qua dòng lệnh. Tạo ra lớp giao diện (Frontend) sẽ giúp người dùng nhập câu hỏi tự nhiên và nhận câu trả lời trực quan.
- **Khả năng mở rộng:** Việc tách biệt giữa file mô hình và file ứng dụng giúp dễ dàng cập nhật. Khi có dữ liệu mới, chỉ cần huấn luyện lại và thay thế file mô hình mà không cần sửa đổi mã nguồn của ứng dụng web.

5.2 Giới thiệu các công nghệ đóng gói và triển khai ứng dụng

Để chuyển đổi mô hình học máy từ môi trường huấn luyện sang một ứng dụng tương tác thực tế, đồ án sử dụng hai công nghệ chủ chốt là **Joblib** (cho việc đóng gói/lưu trữ) và **Streamlit** (cho việc xây dựng giao diện).

5.2.1 Joblib

Joblib là một thư viện mã nguồn mở trong Python, được thiết kế đặc biệt để cung cấp các công cụ hỗ trợ tính toán song song và lưu trữ dữ liệu hiệu quả. Trong lĩnh vực Học máy (Machine Learning), Joblib đóng vai trò quan trọng trong quá trình **Tuần tự hóa (Serialization)**.

Vai trò: Joblib được sử dụng để "đóng gói" toàn bộ trạng thái của hệ thống sau khi huấn luyện vào một tệp nhị phân duy nhất (.pkl). Các thành phần được đóng gói bao gồm:

- Mô hình thuật toán đã được huấn luyện.
- Bộ chuyển đổi đặc trưng (TF-IDF) chứa từ điển vựng.

- Cơ sở dữ liệu ánh xạ câu trả lời (Response Map).

5.2.2 Streamlit

Streamlit là một framework mã nguồn mở cho phép các kỹ sư dữ liệu và chuyên gia máy học xây dựng các ứng dụng web tương tác đẹp mắt chỉ bằng ngôn ngữ **Python**, mà không cần kiến thức chuyên sâu về lập trình web (HTML, CSS, JavaScript).

Đặc điểm nổi bật:

- **Pure Python:** Mọi thành phần giao diện (nút bấm, ô nhập liệu, hiển thị văn bản) đều được định nghĩa bằng mã Python đơn giản.
- **Cơ chế Reactive:** Streamlit tự động chạy lại mã nguồn từ trên xuống dưới mỗi khi người dùng tương tác với giao diện, giúp cập nhật trạng thái ứng dụng tức thì.
- **Hỗ trợ Visualization:** Tích hợp sẵn các công cụ hiển thị dữ liệu phong phú (biểu đồ, bảng, markdown).
- **Dễ dàng triển khai:** Tương thích tốt với các nền tảng đám mây, giúp đưa mô hình lên môi trường internet nhanh chóng.

Vai trò: Streamlit đóng vai trò là lớp giao diện người dùng (Frontend), thực hiện các chức năng:

- Tạo giao diện Chatbot
- Tải mô hình đã đóng gói từ file .pkl vào bộ nhớ.
- Tiếp nhận câu hỏi từ người dùng, gọi mô hình xử lý và hiển thị câu trả lời lên màn hình.

5.3 Quy trình thực hiện

Giai đoạn 1: Đóng gói mô hình

Gom nhóm 3 thành phần cốt lõi của hệ thống vào một đối tượng từ điển (dictionary):

- **Model:** Thuật toán (Naive Bayes) đã học được các trọng số và xác suất từ dữ liệu.
- **Vectorizer (TF-IDF):** Bộ từ điển giúp biến đổi văn bản tiếng Việt thành vector số (giữ nguyên bộ từ vựng đã học).
- **Response Map:** Cơ sở dữ liệu ánh xạ giữa Nhãn ý định (Intent ID) và Câu trả lời mẫu (Response Text).

Sử dụng thư viện joblib để nén đối tượng trên thành một tệp nhị phân duy nhất (chatbot_data.pkl). Tệp này chứa toàn bộ "trí tuệ" của Chatbot.

```python
import joblib
# Đóng gói 3 thứ quan trọng nhất vào 1 file:
# 1. Model Bigram (Bộ não)
# 2. Vectorizer Bigram (Bộ dịch)
# 3. Response Map (Bộ câu trả lời)
data_to_save = {
    "model": model_bi,              # Model
    "vectorizer": vectorizer_bi,    # Vectorizer tương ứng
    "response_map": response_map    # Từ điển câu trả lời
}

# Lưu vào file
with open("chatbot_data.pkl", "wb") as f:
    joblib.dump(data_to_save, f)

print("Đã lưu model thành công vào file chatbot_data.pkl")
```
✓ 0.0s

```
Đã lưu model thành công vào file chatbot_data.pkl
```

Hình 34. Đóng gói mô hình

Giai đoạn 2: Vận hành

Đây là quy trình diễn ra khi chạy ứng dụng Streamlit.

Khởi động hệ thống:

- Khởi chạy Streamlit.
- Hệ thống đọc file chatbot_data.pkl và khôi phục lại 3 thành phần (Model, Vectorizer, Response Map) vào bộ nhớ RAM.

```python
# 2. LOAD MODEL ĐÃ LƯU

@st.cache_resource
def load_data():
    try:
        with open("chatbot_data.pkl", "rb") as f:
            data = joblib.load(f)
        return data["model"], data["vectorizer"], data["response_map"]
    except FileNotFoundError:
        st.error("Lỗi: Không tìm thấy file chatbot_data.pkl")
        return None, None, None

model, vectorizer, response_map = load_data()
```

Hình 35. Load model

Luồng xử lý phần hỏi đáp:

- **Tiếp nhận:** Người dùng nhập câu hỏi (Input Text) qua giao diện chat.

- **Tiền xử lý và Vector hóa:** Hệ thống áp dụng quy trình chuẩn hóa văn bản giống như lúc huấn luyện: Chuyển chữ thường -> Tách từ -> Loại bỏ stopwords. Và sử dụng Vectorizer để chuyển câu hỏi đã xử lý thành vector số.

```
# a. Xử lý đầu vào
processed_text = processing_pipeline(prompt)
input_vec = vectorizer.transform([processed_text])
```

Hình 36. Xử lý đầu vào

- **Dự đoán:** Mô hình tính toán xác suất của câu hỏi đối với từng nhóm ý định và lấy ra nhãn có xác suất cao nhất và giá trị xác suất đó.

```
# b. Tính xác suất
probs = model.predict_proba(input_vec)[0]  # Lấy danh sách xác suất
max_prob = max(probs)                      # Lấy độ tin cậy cao nhất
pred_index = probs.argmax()                # Lấy vị trí index
pred_intent = model.classes_[pred_index]   # Lấy tên nhãn (Tag)
```

Hình 37. Tính xác suất

- **Kiểm tra ngưỡng:**
  - Nếu độ tin cậy > 0.15 (15%): Hệ thống tra cứu Response Map theo nhãn dự đoán và trả về câu trả lời tương ứng.
  - Nếu độ tin cậy <= 0.15: Hệ thống trả về câu phản hồi mặc định ("Xin lỗi, câu này mình chưa được học...") để tránh cung cấp thông tin sai lệch.

```
# c. Kiểm tra ngưỡng tin cậy (Threshold)
if max_prob > THRESHOLD:
    # Nếu tin cậy > 15% -> Tìm câu trả lời
    if pred_intent in response_map:
        response_text = response_map[pred_intent][0]
    else:
        response_text = "Xin lỗi, mình gặp lỗi dữ liệu (có nhãn nhưng thiếu câu trả lời)."
else:
    # Nếu tin cậy <= 15% -> Trả lời không biết
    response_text = "Xin lỗi, câu này mình chưa được học hoặc bạn nhập chưa rõ nghĩa. Bạn thử diễn đạt lại xem?"
```

Hình 38. Kiểm tra ngưỡng tin cậy

- **Phản hồi:** Hiển thị câu trả lời lên giao diện màn hình.

```
# Hiển thị câu trả lời của Bot
with st.chat_message("assistant"):
    st.markdown(response_text)
st.session_state.messages.append({"role": "assistant", "content": response_text})
```

Hình 39. Hiển thị câu trả lời

CHƯƠNG 6: TỔNG KẾT

6.1 Kết quả đạt được

Nhóm đã xây dựng hoàn thiện Chatbot tư vấn quy chế đào tạo cho sinh viên HUMG với các kết quả chính:

- **Xử lý dữ liệu:** Chuẩn hóa dữ liệu JSON và tiền xử lý tiếng Việt hiệu quả (tách từ bằng pyvi, loại bỏ stopwords).
- **Mô hình:** Áp dụng TF-IDF + Bigram và thuật toán Naive Bayes, đạt độ chính xác khoảng 85%.
- **Tối ưu hóa phản hồi:** Thiết lập ngưỡng tin cậy (Threshold = 0.15) để loại bỏ các câu hỏi không rõ nghĩa, giảm thiểu trả lời sai.
- **Ứng dụng:** Đóng gói mô hình thành công và triển khai giao diện web trực quan qua Streamlit.

6.2 Hạn chế

- **Dữ liệu:** Chưa bao phủ hết các biến thể ngôn ngữ (từ viết tắt, không dấu), mẫu câu chưa đủ đa dạng.
- **Thuật toán:** Naive Bayes đôi khi bị phân vân bởi các từ khóa phổ biến (như "điểm", "học phần"), dẫn đến độ tin cậy thấp ở một số câu hỏi phức tạp.

6.3 Hướng phát triển

- **Dữ liệu:** Tăng cường mẫu câu cho các chủ đề khó. Thu thập dữ liệu của người dùng qua các phản hồi.
- **Thuật toán:** Thử nghiệm các thuật toán phức tạp hơn.
- **Mở rộng:** Tích hợp Chatbot lên các nền tảng mạng xã hội phổ biến.

TÀI LIỆU THAM KHẢO

Data Mining | Coursera

Text Preprocessing - Kỹ Thuật Tiền Xử Lý Văn Bản Trong NLP

Xây dựng chương trình tóm tắt văn bản (tiếng Việt) đơn giản với Machine Learning

Machine Learning cơ bản

Thuật toán phân lớp Naive Bayes

Đánh giá các mô hình học máy

Joblib: How to Save and Load Machine Learning Models in Python

Deploying Machine Learning Models with Python & Streamlit – 365 Data Science